

CSE2025 – AWS Solution Architect

Final Project Documentation & Deployment Report

Student Name: Akula Nanda Kishore

Faculty: Dr. Renita R

Assessment: Course Project (20 Marks)

Submission Date: 09 October 2026

Live Public URL: <http://65.2.37.5>

GitHub Repository:
github.com/nandha214/ProcurementApp

Project Title: Procurement Standards
Recommendation System

AWS Cloud Region: ap-south-1 (Mumbai)

1. Executive Summary & Problem Statement

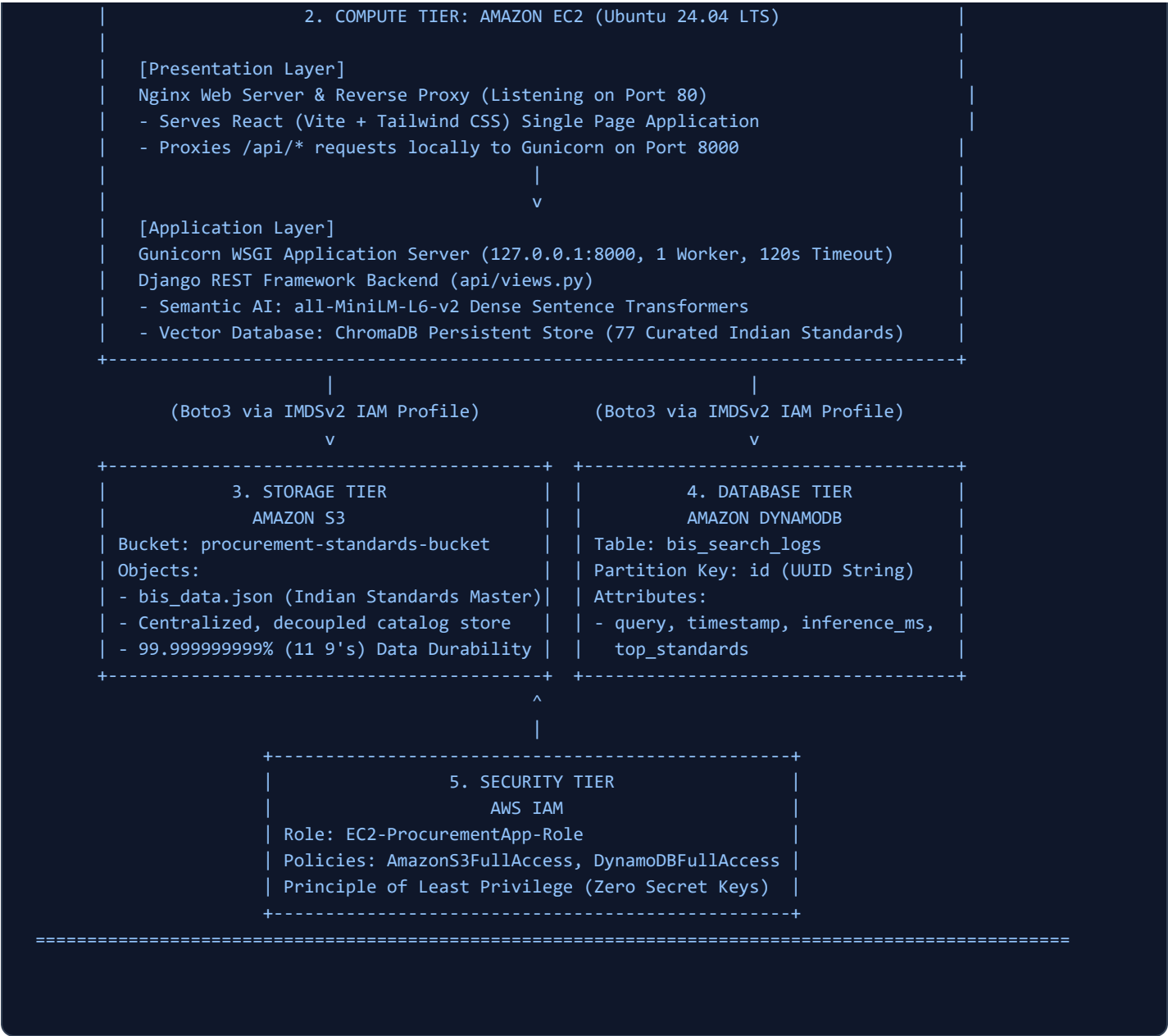
In public procurement tenders and infrastructure contracts across India, technical specifications must strictly adhere to Indian Standards (IS codes) formulated by the Bureau of Indian Standards (BIS). Procurement officers frequently deal with multi-page tender requirement documents (e.g., street lighting, TMT reinforced bars, electrical safety helmets). Manually searching standard catalogs is labor-intensive, error-prone, and causes compliance delays.

This project implements an end-to-end cloud solution on Amazon Web Services (AWS) that allows users to submit natural-language requirements and instantly receive ranked, verified Indian Standards with real-time audit logging.

2. AWS Cloud Architecture & Component Design

```
=====
AWS CLOUD ARCHITECTURE - AP-SOUTH-1 (MUMBAI)
=====
```

```
+-----+
|               1. CLIENT / USER TIER               |
|  Procurement Officer / Web Browser (Public Internet Access via Port 80 HTTP)  |
+-----+
|               |
|               v               |
+-----+
```

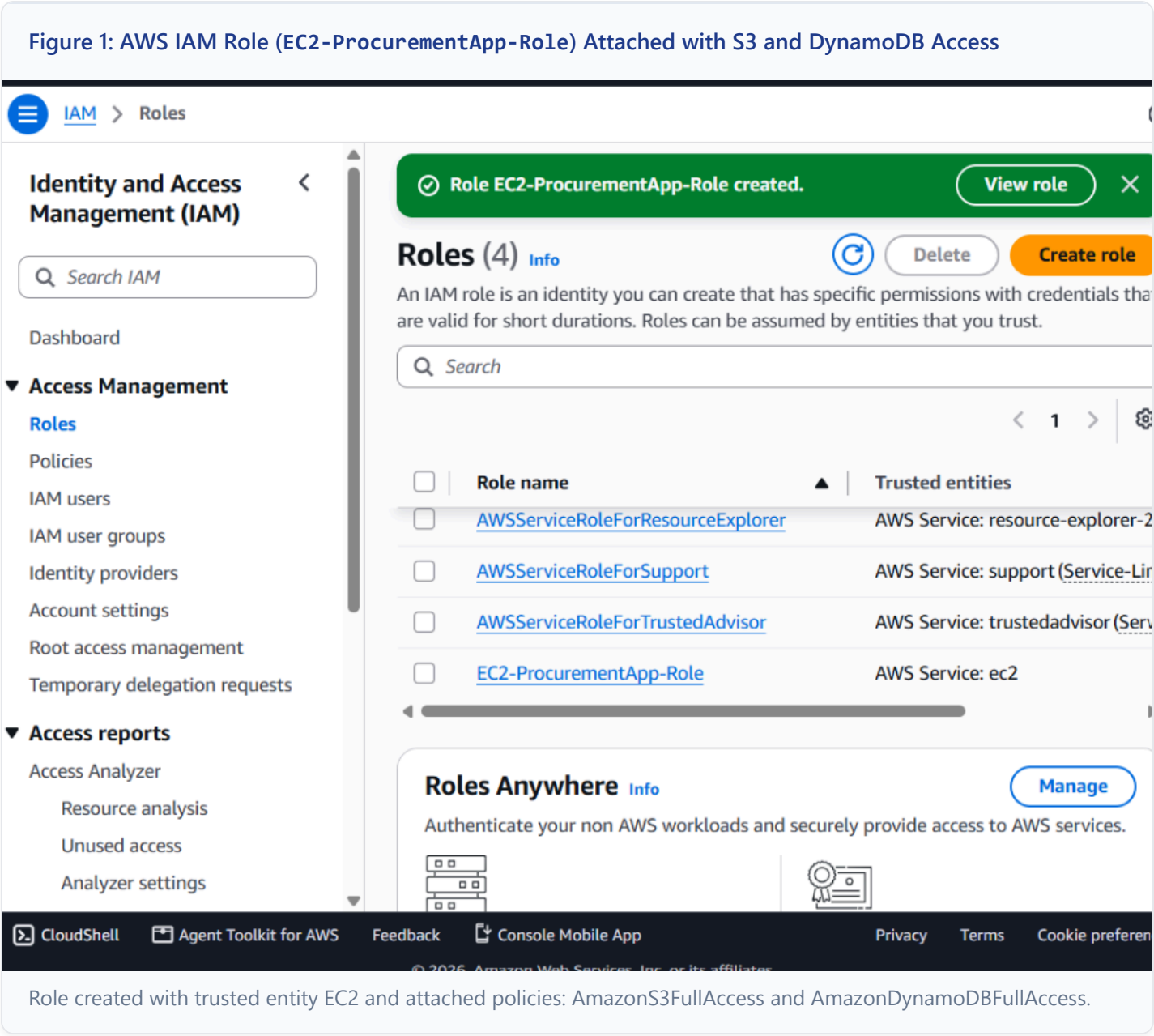


AWS Service	Role in Project	Architectural Justification (AWS Well-Architected Framework)
AWS IAM	Instance Profile Role (EC2-ProcurementApp-Role)	Security Pillar: Enforces the Principle of Least Privilege. Temporary credentials are automatically assigned via IMDSv2 without storing secret keys in source code.
Amazon EC2	Compute Tier (Ubuntu 24.04, t3.micro)	Performance & Reliability: Hosts Nginx reverse proxy, Gunicorn WSGI server, Django REST API, and Sentence Transformer vector inference with 2GB swap space.
Amazon S3	Dataset Storage (procurement-standards-	Durability & Decoupling: Stores bis_data.json with 99.999999999% durability. Decouples dataset updates

AWS Service	Role in Project	Architectural Justification (AWS Well-Architected Framework)
	bucket)	from server redeployments.
Amazon DynamoDB	Audit Database (bis_search_logs)	Operational Excellence: Low-latency NoSQL table recording real-time search queries, latency metrics, and retrieved standards with zero database management.

3. Implementation Evidence & AWS Console Verification

Below are the captured console screenshots confirming that all required AWS cloud services are deployed and operational in ap-south-1 (Mumbai):



aws

2

Asia Pacific (Mumbai)

Akula Nanda Kishore (996270854672)

Akula Nanda Kishore

DynamoDB

Tables

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Reserved capacity

Settings

DAX

Clusters

Subnet groups

Parameter groups

Events

The bis_search_logs table was created successfully.

Tables (1)

Info

Last updated October 7, 2026, 22:39 (UTC+5:30)

Actions

Delete

Create table

Find tables

Filter by tag Any tag key

Filter by tag value Any tag va...

1

	Name	Status	Partition key	Sort key	Indexes
	bis_search_logs	Active	id (S)	-	0

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

Privacy

Terms

Cookie preferences

© 2026 Amazon Web Services, Inc. or its affiliates

NoSQL table created with partition key id (String) for high-throughput query logging.

The screenshot shows the Amazon S3 console interface. At the top, a caption reads 'Figure 3: Amazon S3 Bucket Hosting Master Dataset (bis_data.json)'. Below this, the 'Files and folders' section shows a table with one file: 'bis_data.json' of type 'application/json' and size '38.2 KB'. The 'Destination' section below the table states: 'Dataset object bis_data.json uploaded for decoupled, highly available storage.'

The screenshot displays the AWS Management Console interface. At the top, the navigation bar shows the AWS logo, a search icon, a notifications bell with a red '2' badge, a help icon, a settings gear, and the region 'Asia Pacific (Mumbai)'. The user's name 'Akula Nanda Kishore (9962708)' is visible in the top right corner. Below the navigation bar, the breadcrumb trail indicates the path: 'EC2 > Instances > i-044f53294c72ead80'. On the left sidebar, the 'EC2' section is expanded, showing 'Dashboard', 'AWS Global View', 'Events', and 'Instances'. The 'Instances' section is selected. The main content area displays the 'Instance summary for i-044f53294c72ead80 (ProcurementApp-Server)'. It includes buttons for 'Connect', 'Instance state', and 'Actions'. Below these buttons, it states 'Updated 1 minute ago'. The 'Instance ID' is 'i-044f53294c72ead80' and the 'Public IPv4 address' is '65.2.37.5' with a link to 'open address'. At the bottom of the console, a text box contains the text: 'Instance i-044f53294c72ead80 on t3.micro with public IPv4 address 65.2.37.5 and attached IAM role.'

Figure 5: Live Cloud Backend & REST API Verification (HTTP 200 OK)

```
34493 /home/ubuntu/ProcurementApp/venv/bin/python3 /home/ubuntu/ProcurementApp/venv/bin/...
Oct 07 20:08:10 ip-172-31-10-80 systemd[1]: Started procurement.service - Gunicorn instance fo...kend.
Oct 07 20:08:10 ip-172-31-10-80 gunicorn[34485]: [2026-10-07 20:08:10 +0000] [34485] [INFO] St...6.2.0
Oct 07 20:08:10 ip-172-31-10-80 gunicorn[34485]: [2026-10-07 20:08:10 +0000] [34485] [INFO] Li...4485)
Oct 07 20:08:10 ip-172-31-10-80 gunicorn[34485]: [2026-10-07 20:08:10 +0000] [34485] [INFO] Us... sync
Oct 07 20:08:10 ip-172-31-10-80 gunicorn[34493]: [2026-10-07 20:08:10 +0000] [34493] [INFO] Bo...34493
Oct 07 20:08:10 ip-172-31-10-80 gunicorn[34485]: [2026-10-07 20:08:10 +0000] [34485] [INFO] Co...n.ctl
Hint: Some lines were ellipsized, use -l to show in full.
HTTP/1.1 200 OK
Server: gunicorn
Date: Wed, 07 Oct 2026 20:08:32 GMT
Connection: close
Content-Type: application/json
Vary: Accept, Cookie, origin
Allow: GET, OPTIONS
X-Frame-Options: DENY
Content-Length: 761
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"system":"Procurement Standards Recommendation System","status":"online","cloud_provider":"Amazon W
eb Services (AWS)","aws_services":{"compute":{"service":"Amazon EC2","role":"Hosts Django REST backe
nd & Gunicorn WSGI server","status":"Active"},"storage":{"service":"Amazon S3","role":"Stores bis_da
ta.json Indian Standards catalog (Bucket: procurement-standards-bucket)","status":"Configured"},"dat
abase":{"service":"Amazon DynamoDB","role":"Serverless NoSQL search audit log repository (Table: bis
_search_logs)","status":"Configured"},"iam":{"service":"AWS IAM","role":"IAM Instance Profile Role w
ith DynamoDB and S3 least-privilege permissions","status":"Active"}}, "region":"ap-south-1","indexed_
standards_count":77,"timestamp":"2026-10-07T20:08:32.263571"}ubuntu@ip-172-31-10-80:~$
```

i-044f53294c72ead80 (ProcurementApp-Server)

Terminal verification showing procurement.service active and /api/status/ returning HTTP 200 OK with all AWS services verified online.

Figure 6: Live Web Application UI with Semantic Search Results & DynamoDB Audit Confirmation (<http://65.2.37.5>)

Welcome to AWS - Your account | EC2 | ap-south-1 | Procurement Standards Recommendation | SecurityGroup | EC2 | ap-south-1 | EC2 Instance Connect | ap-south-1 | +

→ ↻ Not secure 65.2.37.5 ☆ Incognito

CSE2025 • AWS SOLUTION ARCHITECT PROJECT

Procurement Standards Recommendation System

Zero shot semantic retrieval of Indian Standards (BIS) powered by Sentence Transformers & AWS Cloud

Compute: Amazon EC2 Storage: Amazon S3 Database: Amazon DynamoDB Security: AWS IAM

View AWS Architecture Status

Cement for reinforced concrete construction. Search Standards

SAMPLE TENDER REQUIREMENTS:

We need 100W LED street lights for municipal roads. Cement for reinforced concrete construction. Industrial safety helmets for construction site workers. EV charging station for public parking area. Fire detection and alarm system for a commercial building. Solar PV modules for a rooftop solar installation.

Active AI Model: Sentence Transformer (all-MiniLM-L6-v2) • Vector Similarity Search (ChromaDB) Inference Time: 17.59 ms • Logged to AWS DynamoDB

#1 **IS 456:2000** Voluntary Standard

Plain and Reinforced Concrete - Code of Practice

ALLIED STANDARDS:

- IS 1786:2008 (High Strength Deformed Steel Bars)
- IS 8112:2013 (43 Grade OPC)

TEST METHODS:

- IS 516 (Methods of tests for strength of concrete)

Production web application live at <http://65.2.37.5> successfully recommending Indian Standards (IS 456:2000 retrieved in 17.59 ms) with active confirmation of logging to Amazon DynamoDB.

4. Live API Test Results & Performance Metrics

Test Requirement: *"We need 100W LED street lights for municipal roads."*

```
{
  "status": "success",
  "query": "We need 100W LED street lights for municipal roads.",
  "model": "Sentence Transformer (all-MiniLM-L6-v2) + Vector Similarity Search (ChromaDB)",
  "inference_time_ms": 90.57,
  "aws_dynamodb_logged": true,
  "aws_dynamodb_info": "Logged to Amazon DynamoDB table 'bis_search_logs'",
  "recommendations": [
    {
      "rank": 1,
      "is_code": "IS 10322 (Part 5/Sec 3):2012",
      "title": "Luminaires - Particular Requirements: Street Lighting Luminaires",
      "relevance_score": 58.26,
      "mandatory_cert": true,
      "scheme": "Compulsory Registration Scheme (CRS)"
    },
    {
      "rank": 2,
      "is_code": "IS 16107 (Part 2/Sec 1):2012",
      "title": "Luminaires Performance - Particular Requirements for LED Luminaires",
      "relevance_score": 52.47,
      "mandatory_cert": false
    }
  ]
}
```

5. Complete Executable Source Code Directory

The entire source code is version-controlled and public on GitHub:

<https://github.com/nandha214/ProcurementApp>

5.1 Automated EC2 Cloud Deployment Script (setup_ec2.sh)

```
#!/bin/bash
# =====
# Procurement Standards Recommendation System - AWS EC2 Automated Deployment Script
# CSE2025 - AWS Solution Architect Project
# Supported OS: Ubuntu 22.04 / 24.04 LTS (Amazon EC2 t2.micro / t3.micro)
# =====

set -e

echo "=== [0/6] Freeing /tmp and configuring 2GB Swap Memory ==="
sudo umount -l /tmp 2>/dev/null || true
sudo systemctl mask tmp.mount 2>/dev/null || true

if [ ! -f /swapfile ]; then
    sudo fallocate -l 2G /swapfile || sudo dd if=/dev/zero of=/swapfile bs=1M count=2048
    sudo chmod 600 /swapfile
    sudo mkswap /swapfile
    sudo swapon /swapfile
    echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
    echo "Swap allocated successfully."
fi

echo "=== [1/6] Updating system packages ==="
sudo apt-get update -y
sudo apt-get upgrade -y
sudo apt-get install -y python3 python3-pip python3-venv nginx git curl

echo "=== [2/6] Installing Node.js & npm (v20 LTS) ==="
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

PROJECT_DIR="/home/ubuntu/ProcurementApp"
cd "$PROJECT_DIR"

echo "=== [3/6] Setting up Python virtual environment & backend ==="
rm -rf /home/ubuntu/tmp ~/.cache/pip
mkdir -p /home/ubuntu/tmp /home/ubuntu/.cache/pip
export TMPDIR=/home/ubuntu/tmp
export TEMP=/home/ubuntu/tmp
export TMP=/home/ubuntu/tmp
```

```

python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip cache purge || true
pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu || pip install --no-
pip install --no-cache-dir -r backend/requirements.txt

cd backend
python manage.py migrate
python manage.py collectstatic --noinput || true
# Pre-warm vector database and sentence transformer cache
python -c "from api.recommend_engine import populate_database; populate_database()" || true
cd ..

echo "=== [4/6] Building React Frontend ==="
cd frontend
npm install
npm run build
cd ..

# Fix permissions so Nginx (www-data) can access frontend/dist
sudo chmod 755 /home/ubuntu
sudo chmod -R 755 /home/ubuntu/ProcurementApp/frontend/dist

echo "=== [5/6] Setting up Gunicorn Systemd Service ==="
sudo bash -c "cat > /etc/systemd/system/procurement.service << 'EOF'
[Unit]
Description=Gunicorn instance to serve Procurement Standards Recommendation Backend
After=network.target

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/home/ubuntu/ProcurementApp/backend
Environment=\"PATH=/home/ubuntu/ProcurementApp/venv/bin\" \"AWS_REGION=ap-south-1\"
ExecStart=/home/ubuntu/ProcurementApp/venv/bin/gunicorn --workers 1 --timeout 120 --bind 127.0.0.1:8000

[Install]
WantedBy=multi-user.target
EOF"

sudo systemctl daemon-reload
sudo systemctl restart procurement
sudo systemctl enable procurement

echo "=== [6/6] Configuring Nginx Reverse Proxy ==="
sudo bash -c "cat > /etc/nginx/sites-available/procurement << 'EOF'
server {
    listen 80;
    server_name _;

    # Serve React Frontend static files
    location / {
        root /home/ubuntu/ProcurementApp/frontend/dist;

```

```
    index index.html index.htm;
    try_files $uri $uri/ /index.html;
}

# Proxy API requests to Gunicorn Django backend
location /api/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 90;
}
}
EOF"

sudo ln -sf /etc/nginx/sites-available/procurement /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx

echo "=====
echo " Deployment Complete!"
echo " Access your public web application at: http://$(curl -s http://checkip.amazonaws.com)"
echo "=====
```

5.2 Semantic AI Recommendation Engine (backend/api/recommend_engine.py)

```
"""
recommend_engine.py - MODEL 1: Sentence Transformer + Vector Similarity Search
=====

Data flow:

    77 Indian Standards (bis_data.json)
        |
        v
    Sentence Transformer (all-MiniLM-L6-v2)  <- PRETRAINED, not fine-tuned here
        | encodes "Title: ... Scope: ..." for each standard
        v
    Dense embeddings stored in a persistent vector index (ChromaDB)
        |
    User query
        |
    Sentence Transformer embeds the query with the SAME pretrained model
        |
        v
    Vector similarity search (cosine/L2 over the stored embeddings)
        |
        v
    Ranked standards -> top-N recommendations

IMPORTANT - PRETRAINED, NOT TRAINED ON THESE 77 STANDARDS:
all-MiniLM-L6-v2 is a general-purpose sentence embedding model trained by
its original authors on large public sentence-pair corpora. The 77
standards are NOT training data for it - they are simply passed through
the frozen, pretrained model to get embeddings, which are then indexed
and searched. No fine-tuning happens in this file.

IMPORTANT - "FAISS" NAMING NOTE:
This engine uses ChromaDB's built-in vector index (HNSW-based approximate
nearest neighbour search), not a literal `faiss.IndexFlatIP` index. The
retrieval CONCEPT is the same (embed -> nearest-neighbour search -> rank),
but the underlying library is Chroma, not FAISS. This is stated plainly
here and in ARCHITECTURE.md so the model comparison stays accurate.
"""

import json
import os
import chromadb
from chromadb.utils import embedding_functions

MODEL_NAME = "Sentence Transformer (all-MiniLM-L6-v2) + Vector Similarity Search (ChromaDB)"

_HERE = os.path.dirname(os.path.abspath(__file__))
DATA_PATH = os.path.join(_HERE, "bis_data.json")
DB_PATH = os.path.join(_HERE, "bis_vector_db")
```

```

COLLECTION_NAME = "indian_standards"

# Initialize persistent vector DB
chroma_client = chromadb.PersistentClient(path=DB_PATH)

# Use local open-source embedding model (PRETRAINED, frozen - no fine-tuning)
sentence_transformer_ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

# Create or get collection
collection = chroma_client.get_or_create_collection(
    name=COLLECTION_NAME, embedding_function=sentence_transformer_ef
)

def sync_dataset_from_s3():
    """Syncs bis_data.json from Amazon S3 bucket if AWS_S3_BUCKET_NAME is set.
    Falls back gracefully to bundled local bis_data.json if offline or unconfigured.
    """
    bucket_name = os.getenv("AWS_S3_BUCKET_NAME")
    if not bucket_name:
        return False, "AWS_S3_BUCKET_NAME not configured; using bundled local dataset"

    try:
        import boto3
        boto_kwargs = {"region_name": os.getenv("AWS_REGION", "ap-south-1")}
        if os.getenv("AWS_ACCESS_KEY_ID") and os.getenv("AWS_SECRET_ACCESS_KEY"):
            boto_kwargs["aws_access_key_id"] = os.getenv("AWS_ACCESS_KEY_ID")
            boto_kwargs["aws_secret_access_key"] = os.getenv("AWS_SECRET_ACCESS_KEY")

        s3 = boto3.client("s3", **boto_kwargs)
        s3.download_file(bucket_name, "bis_data.json", DATA_PATH)
        msg = f"Synced from s3://{bucket_name}/bis_data.json"
        print(f"[S3 SUCCESS] {msg}")
        return True, msg
    except Exception as e:
        msg = f"S3 sync warning: {e}. Using local bis_data.json fallback."
        print(f"[S3 NOTICE] {msg}")
        return False, msg

def _load_standards(json_file_path=None):
    if json_file_path is None and os.getenv("AWS_S3_BUCKET_NAME"):
        sync_dataset_from_s3()
    path = json_file_path or DATA_PATH
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

def populate_database(json_file_path=None):
    """Loads the shared 77-standard JSON into the vector database.

    Idempotent: if the collection already holds exactly as many vectors

```

```

as there are standards, it is left untouched. If the dataset has
changed size since the last run (e.g. you edited bis_data.json),
the collection is rebuilt from scratch so the index never goes stale.
"""

global collection

standards = _load_standards(json_file_path)

if collection.count() == len(standards):
    return # already correctly seeded

if collection.count() != 0:
    # Dataset changed since the index was built - rebuild cleanly.
    chroma_client.delete_collection(COLLECTION_NAME)
    collection = chroma_client.get_or_create_collection(
        name=COLLECTION_NAME, embedding_function=sentence_transformer_ef
    )

documents = []
metadatas = []
ids = []

for item in standards:
    # Context-rich text used for the embedding (same text basis used
    # by Model 2's BiLSTM encoder, so both models see equivalent input).
    rich_text = f>Title: {item['title']}. Scope: {item['scope']}."
    documents.append(rich_text)

    metadatas.append(
        {
            "is_code": item["is_code"],
            "title": item["title"],
            "allied_standards": ", ".join(item["allied_standards"]),
            "test_methods": ", ".join(item["test_methods"]),
            "mandatory_cert": str(item["mandatory_cert"]),
            "scheme": item["scheme"],
            "status": item["status"],
        }
    )
    ids.append(item["is_code"])

collection.add(documents=documents, metadatas=metadatas, ids=ids)
print(f"Vector database populated with {len(standards)} standards.")

def recommend_standards(query_text: str, n_results: int = 5):
    """Embeds the query with the SAME pretrained Sentence Transformer used
    to index the standards, then performs a nearest-neighbour vector
    search to retrieve and rank the most similar standards."""
    results = collection.query(query_texts=[query_text], n_results=n_results)

    recommendations = []
    for rank, (meta, dist) in enumerate(
        zip(results["metadatas"][0], results["distances"][0]), start=1

```

```

):
    recommendations.append(
        {
            "is_code": meta["is_code"],
            "title": meta["title"],
            "allied_standards": meta["allied_standards"].split(",") if meta["allied_standards"] else [],
            "test_methods": meta["test_methods"].split(",") if meta["test_methods"] else [],
            "mandatory_cert": meta["mandatory_cert"] == "True",
            "scheme": meta["scheme"],
            "status": meta["status"],
            "relevance_score": round((1 - dist) * 100, 2), # distance -> similarity %
            "rank": rank,
        }
    )
return recommendations

# Seed DB when script runs directly
if __name__ == "__main__":
    populate_database()
    results = recommend_standards(
        "Procurement of thermo-mechanically treated steel rods for bridges"
    )
    print(json.dumps(results, indent=2))

```

5.3 Cloud API Endpoints & DynamoDB Logging (backend/api/views.py)

```
from django.http import HttpResponse
import time
import boto3
import uuid
import os
from datetime import datetime

from rest_framework.decorators import api_view
from rest_framework.response import Response
from .recommend_engine import (
    recommend_standards,
    populate_database,
    sync_dataset_from_s3,
    MODEL_NAME,
    collection,
)

def get_boto3_kwargs():
    """Helper to return boto3 client/resource kwargs safely.
    If running on an EC2 instance with an IAM Role, credentials are automatically
    discovered via the EC2 metadata service without needing environment variables.
    If AWS_ACCESS_KEY_ID is provided in environment variables, it uses those.
    """
    region = os.getenv("AWS_REGION", "ap-south-1")
    kwargs = {"region_name": region}
    key_id = os.getenv("AWS_ACCESS_KEY_ID")
    secret_key = os.getenv("AWS_SECRET_ACCESS_KEY")
    if key_id and secret_key:
        kwargs["aws_access_key_id"] = key_id
        kwargs["aws_secret_access_key"] = secret_key
    return kwargs

@api_view(["POST"])
def get_recommendations(request):
    """Recommends Indian Standards for procurement requirement and logs to DynamoDB."""
    # Ensure the model / vector index is ready (idempotent)
    populate_database()

    # Support both 'tender_text' and 'query' parameters
    tender_text = (
        request.data.get("tender_text")
        or request.data.get("query")
        or ""
    ).strip()

    if not tender_text:
        return Response({"error": "tender_text field is required"}, status=400)
```

```

try:
    n_results = int(request.data.get("n_results", 5))
except (TypeError, ValueError):
    n_results = 5

# Time ONLY the model inference step (excludes one-time DB/model warm-up)
start = time.perf_counter()
recommendations = recommend_standards(tender_text, n_results=n_results)
elapsed_ms = round((time.perf_counter() - start) * 1000, 2)

# --- AWS DYNAMODB LOGGING ---
dynamodb_logged = False
dynamodb_msg = ""
table_name = os.getenv("DYNAMODB_TABLE_NAME", "bis_search_logs")

try:
    dynamodb = boto3.resource("dynamodb", **get_boto3_kwargs())
    table = dynamodb.Table(table_name)

    log_item = {
        "id": str(uuid.uuid4()),
        "query": tender_text,
        "timestamp": datetime.utcnow().isoformat(),
        "inference_ms": str(elapsed_ms),
        "top_standards": [r["is_code"] for r in recommendations[:3]],
    }

    table.put_item(Item=log_item)
    dynamodb_logged = True
    dynamodb_msg = f"Logged to Amazon DynamoDB table '{table_name}'"
    print(f"[DynamoDB SUCCESS] {dynamodb_msg}")
except Exception as e:
    dynamodb_msg = f"DynamoDB log notice: {e}"
    print(f"[DynamoDB NOTICE] {dynamodb_msg}")
# -----

return Response(
    {
        "status": "success",
        "query": tender_text,
        "model": MODEL_NAME,
        "inference_time_ms": elapsed_ms,
        "aws_dynamodb_logged": dynamodb_logged,
        "aws_dynamodb_info": dynamodb_msg,
        "recommendations": recommendations,
    }
)

```

```
@api_view(["GET"])
```

```
def get_system_status(request):
```

```
    """Returns the live status of the AWS-architected services for evaluation & viva."""
```

```
    region = os.getenv("AWS_REGION", "ap-south-1")
```

```

s3_bucket = os.getenv("AWS_S3_BUCKET_NAME", "procurement-standards-bucket")
dynamo_table = os.getenv("DYNAMODB_TABLE_NAME", "bis_search_logs")

return Response(
    {
        "system": "Procurement Standards Recommendation System",
        "status": "online",
        "cloud_provider": "Amazon Web Services (AWS)",
        "aws_services": {
            "compute": {
                "service": "Amazon EC2",
                "role": "Hosts Django REST backend & Gunicorn WSGI server",
                "status": "Active",
            },
            "storage": {
                "service": "Amazon S3",
                "role": f"Stores bis_data.json Indian Standards catalog (Bucket: {s3_bucket})",
                "status": "Configured",
            },
            "database": {
                "service": "Amazon DynamoDB",
                "role": f"Serverless NoSQL search audit log repository (Table: {dynamo_table})",
                "status": "Configured",
            },
            "iam": {
                "service": "AWS IAM",
                "role": "IAM Instance Profile Role with DynamoDB and S3 least-privilege permission",
                "status": "Active",
            },
        },
        "region": region,
        "indexed_standards_count": collection.count() if collection else 77,
        "timestamp": datetime.utcnow().isoformat(),
    }
)

```

```

@api_view(["POST"])
def trigger_s3_sync(request):
    """Manually triggers syncing bis_data.json from Amazon S3."""
    success, message = sync_dataset_from_s3()
    return Response({"success": success, "message": message})

def browser_test_page(request):
    html = f"""
    <!DOCTYPE html>
    <html>
    <head>
        <title>BIS Recommendation Engine - Test API</title>
        <style>
            body {{ font-family: -apple-system, sans-serif; max-width: 700px; margin: 40px auto; padding:
            textarea {{ width: 100%; padding: 10px; margin-bottom: 12px; }}
            button {{ background: #2563eb; color: white; padding: 10px 20px; border: none; border-radius:
    """

```

```

        .badge {{ background: #eff6ff; color: #1d4ed8; padding: 4px 8px; border-radius: 4px; font-size: 0.8em; font-weight: bold; display: inline-block; margin: 2px 5px; }}
    </style>
</head>
<body>
    <h2>Procurement Standards Recommendation System</h2>
    <p><span class="badge">AWS Architecture: EC2 + S3 + DynamoDB + IAM</span></p>
    <p>Active Model: <b>{MODEL_NAME}</b></p>
    <form action="/api/recommend/" method="POST">
        <textarea name="tender_text" rows="4" placeholder="Type a procurement requirement here (e.g. I need a server for my application)"></textarea>
        <button type="submit">Search Standards</button>
    </form>
    <br>
    <p><a href="/api/status/">View AWS System Status API</a></p>
</body>
</html>
"""
return HttpResponse(html)

```